SE-JEPA: Self-Evolving Latent World Models

Anonymous CVPR submission

Abstract

We propose SE-JEPA, a framework that turns deployment experience into reusable predictive knowledge, enabling world models to learn continually under recurring environmental changes. The method decouples fast within-episode adaptation from cross-episode learning, building predictive capabilities by replaying observed interaction trajectories rather than directly inheriting episode-specific weight updates. SE-JEPA automatically groups predictively compatible trajectories into predictive domains and trains a separate slow predictor for each domain in a fixed representation space. Prediction errors guide experience selection and predictor validation, while current interactions determine which predictor to use for planning. The framework requires neither predefined domain labels nor domain-switch signals and supports both gradient-based and sampling-based planners. Experiments show that SE-JEPA improves test success over AdaJEPA by 15.4–17.1 percentage points on PushObj shapes unseen during pretraining and 15.9–18.7 points on PushT visual shifts. In a repeated-goal stream spanning 14 shape and visual conditions, online success increases from 65.3% to 75.5% over five deployment rounds. Furthermore, with DINO-WM as the backbone, a predictor trained through a single consolidation pass reduces Chamfer distance by 47% on rope instances excluded from learning, without within-episode adaptation. These results demonstrate that SE-JEPA not only corrects current predictions but also enables world models to continually improve subsequent planning. The source code will be made publicly available.

1. Introduction

Latent world models provide an efficient approach to planning from visual observations by predicting action-conditioned state evolution in representation space [2, 18, 22, 42]. However, their performance depends on how well the learned model matches the deployment environment. When object shapes, visual conditions, or dynamics deviate from the training distribution, model predictions may diverge from the actual state evolution, causing planning to rely on inaccurate future states. To address this challenge, AdaJEPA [35] adapts its model after each executed action block but resets these adaptations between episodes. JEPA-TTT [41] extends predictor learning across episodes by accumulating interactions and replaying them. Yet continued updating does not, by itself, yield reusable knowledge. Our diagnostic experiments highlight the limitations of simply retaining adapted weights: On PushObj, inheriting adapted weights reduces success from 62% to 48%. In a separate deployment stream, consecutive fast-update fingerprints show weak directional alignment (mean cosine similarity: 0.018). These observations suggest that parameter corrections fitted to individual interaction segments need not benefit subsequent trajectories. Under recurring conditions, deciding which experiences should be learned jointly and when a learned predictor should be reused therefore remains a central challenge for test-time adaptation.

Building on the above insight, we introduce SE-JEPA, a latent world model framework that enables continual learning across episodes through persistent predictive memory. Inspired by complementary learning systems [23], SE-JEPA separates fast within-episode adaptation from slow cross-episode consolidation. The fast system adapts a temporary copy of the model to the current environment, maintaining responsiveness to local changes. The slow system replays historical interaction trajectories to progressively learn dynamics predictors that can be reused for future goals. Accumulating experience over time, however, also requires addressing compatibility: trajectories collected under different conditions may not be well modeled by a single predictor. To this end, SE-JEPA organizes past experience into self-discovered *predictive domains*. Predictively compatible trajectories are grouped into the same domain, each with its own replay buffer and slow predictor. This allows dynamics knowledge from different environments to accumulate separately, rather than forcing it into a single set of parameters.

Furthermore, SE-JEPA uses its own prediction errors to select which experiences to retain and when to reuse learned predictors. Trajectories that the pretrained model already predicts accurately need not be written directly to memory, whereas those with substantial prediction errors support continued learning within the corresponding predictive domains. A slow predictor becomes eligible for retrieval only after consistently outperforming the pretrained model

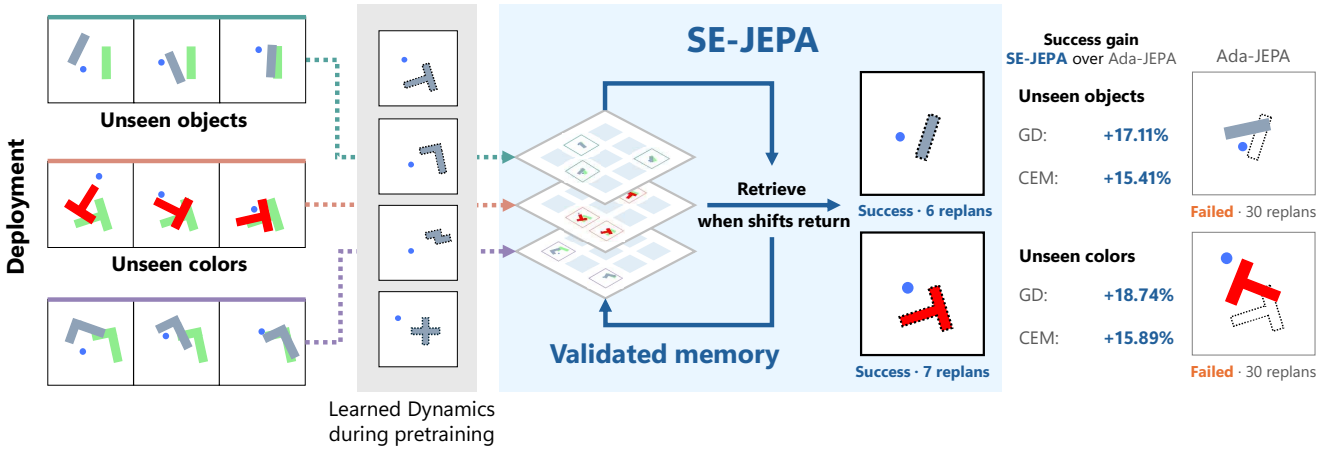


Figure 1. **Deployment experience becomes reusable predictive knowledge.** SE-JEPA consolidates compatible trajectories into validated memory and retrieves learned predictors when conditions recur. The examples contrast successful retrieval with episodic adaptation. Gains over AdaJEPA are percentage-point differences in held-out success on unseen PushObj shapes and PushT visual conditions, with three learning chains per planner.

on new trajectories evaluated before it is trained on them. Thus, in SE-JEPA, historical trajectories establish whether learned knowledge is eligible for reuse, while current interactions determine whether that knowledge is relevant to the present planning problem. This mechanism links experience accumulation to selective reuse, rather than unconditionally applying past adaptations to new interactions.

Our main contributions are summarized as follows:

- We introduce SE-JEPA, which separates fast within-episode adaptation from slow cross-episode consolidation, learning reusable predictors from deployment trajectories rather than accumulating episode-specific weight updates.
- We develop a label-free memory that organizes compatible trajectories into predictive domains. Prediction errors guide experience selection, validate predictors on new trajectories before learning from them, and determine which predictor to reuse for current planning.
- Extensive experiments demonstrate the effectiveness of SE-JEPA in cross-episode knowledge reuse and continued deployment.

2. Related Work

World Models and Representation Learning. World models enable planning with learned dynamics [6, 9, 37], policy learning through imagined rollouts [8, 10–12, 16, 24], tree search [29, 38], and value-guided latent control [13, 14]. In parallel, predictive representation learning [18, 30, 32], non-contrastive self-supervision [3, 7, 40], and visual representation models [5, 25, 26] have enabled learning in increasingly expressive latent spaces. Building on these advances, image and video JEPA [1, 2, 4] and action-conditioned latent world models [22, 36, 39, 42] have advanced representation learning and predictive planning. While these methods focus primarily on learning representations and dynamics, SE-JEPA investigates how deployment experience can be consolidated into reusable predictive knowledge across recurring distribution shifts.

Test-time adaptation. Test-time training and entropy minimization update deployed models from unlabeled observations [31, 33], including under changing distributions [34]. In control, each executed action supplies a target for the dynamics predictor. AdaJEPA [35] uses this signal within an episode and resets the weights at its end. JEPA-TTT [41] studies a dynamics shift that persists across episodes. It trains one predictor by replaying overlapping windows from the accumulated interactions. SE-JEPA addresses recurring shifts. It keeps the pretrained weights fixed, groups compatible trajectories, and retrieves separate slow predictors whose errors on new trajectories are lower than those of the pretrained reference. These comparisons precede replay training on each trajectory.

Continual Learning and Complementary Memory. Complementary learning systems distinguish rapid adaptation from gradual knowledge consolidation [23]. Continual learning mitigates forgetting through experience replay [27], constrained updates [17, 21], knowledge distillation [19], and modular architectures [28]. In world-model planning, however, reusing a predictor directly influences subsequent actions and observations, making knowledge selection as important as retention.

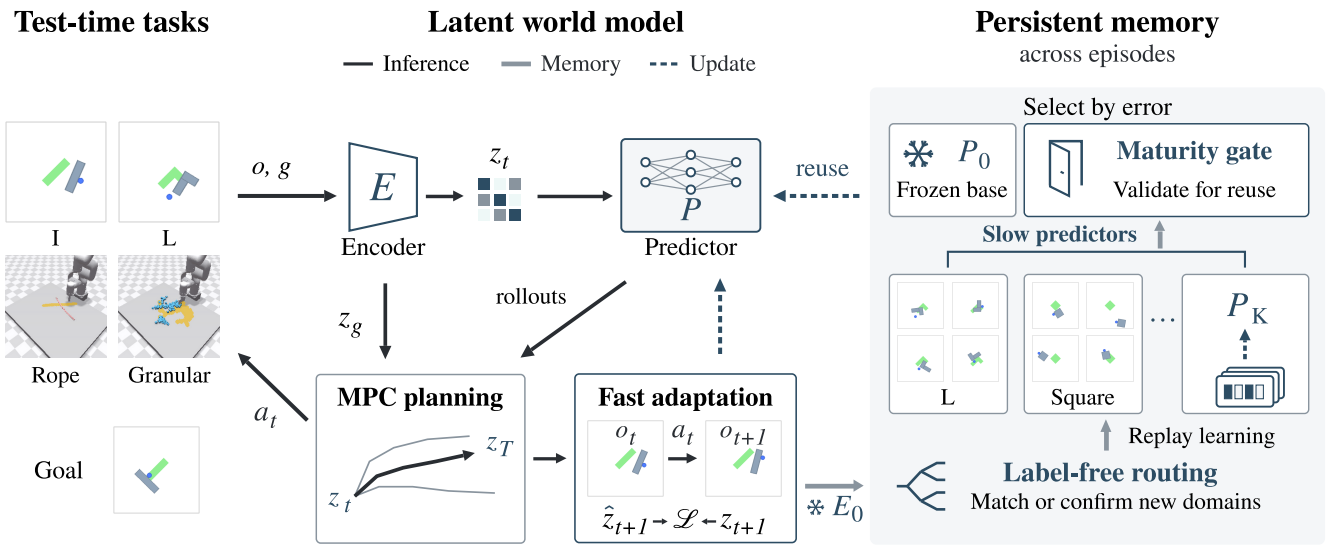


Figure 2. **SE-JEPA architecture.** The fast loop plans in latent space and adapts the predictor from observed transitions. At the end of an episode, the slow system re-encodes the trajectory with the frozen encoder, assigns unfamiliar experience to a self-discovered domain, and trains that domain's slow predictor by replay. Predictors that pass the maturity gate compete with the frozen base at the start of the next episode. Fast parameters reset at every episode; slow predictors and replay persist. Shape names illustrate memory contents and are not supervision. Test-time examples include rigid objects, rope, and granular material.

3. SE-JEPA

3.1. Motivation

An episode-specific correction need not transfer to a new goal, even when the object is unchanged. On 100 matched PushObj goals, inheriting fast weights recovered 11 goals missed by episodic reset but lost 25 that reset solved (Table 3). In a separate seven-round stream, consecutive fast-update fingerprints have mean cosine 0.018 for both GD and CEM over 2,449 pairs per planner. Absolute cosine is below 0.1 for 96.3% and 96.8% of pairs (Appendix D).

Observed transitions preserve the action outcomes themselves: even an unsuccessful push reveals how contact changes an object's pose. SE-JEPA therefore learns across episodes by replaying trajectories in a fixed latent space (Fig. 2). It groups predictively compatible experience without domain labels and uses early interactions to select a predictor for each new goal.

3.2. Fast adaptation and replay consolidation

Each episode starts with working copies of the fixed pretrained encoder E_0 and predictor P_0. The planner minimizes latent goal cost over candidate action sequences [22, 42] and executes the first action block. Following AdaJEPA [35], one Adam step on recent transition errors updates the predictor's last block and encoder head before replanning. These changes last only for the episode. Early replans can also retrieve a slow predictor as an adaptation base (Sec. 3.4).

Across episodes, each *predictive domain* k maintains a replay buffer $\mathcal{R}_k$ and slow predictor P_{ϕ_k}. The frozen encoder re-encodes each accepted trajectory, keeping replay in a consistent latent space despite fast encoder changes. The slow predictor minimizes next-observation error on sampled replay windows:

$$\mathcal{L}_k(\phi_k) = \mathbb{E}_{(x,z^+)\sim\mathcal{R}_k} \left\| P_{\phi_k}(x) - z^+ \right\|_2^2. \tag{1}$$

Here x contains observation embeddings and executed actions from a short history, and z^+ is the next observation embedding. Each slow predictor starts as a separate copy of P_0. Sampling windows across episodes combines different contacts and motions in the frozen representation space.

3.3. Discovering predictive domains

The frozen reference first tests whether an episode contributes new experience. Let $\ell_{e,t}^{(0)}$ be its next-frame prediction error, and $\mathcal{I}_e$ index up to the first three observed transitions. Their geometric mean is

$$F_e = \exp\left(\frac{1}{|\mathcal{I}_e|} \sum_{t\in\mathcal{I}_e} \log \ell_{e,t}^{(0)} \right). \tag{2}$$

If $F_e < \tau_f$, the calibrated familiarity threshold, the agent keeps P_0 as its adaptation base and skips the episode's direct memory write. Replay thus receives episodes the reference cannot yet predict accurately.

For an unfamiliar episode, Δ_e concatenates the fast predictor and encoder-head changes. Its normalized direction

and similarity to domain k are

$$u_e = \frac{\Delta_e}{\|\Delta_e\|_2}, \qquad s(e,k) = u_e^\top c_k, \tag{3}$$

where c_k is the domain's unit-length prototype. The retrieved domain is the first write candidate; otherwise, candidates are ranked by $s(e,k)$. The trajectory is assigned to a candidate domain if its predictor passes the error check in Sec. 3.4. Unassigned trajectories remain in a pending pool. When a new direction aligns with those of two waiting episodes, the three trajectories establish a new domain.

Fingerprints propose candidates; prediction errors decide whether trajectories can share a learner. Delayed creation gathers evidence before fitting a new predictor. Domains merge when their predictors transfer in both directions: the larger domain retains its predictor, combines the buffers, and replays the merged experience. The frozen base remains available. Mature domains can also absorb smaller compatible groups (Appendix A).

3.4. Validated retrieval for planning

A slow predictor becomes eligible for planning by predicting new trajectories accurately before training on them. Before adding trajectory τ_e to a buffer, the agent rolls out the candidate predictor and the pretrained reference under the actions actually taken. Their open-loop errors in the frozen latent space give the ratio

$$r_{e,k} = \frac{\mathcal{L}_{\text{roll}}(\phi_k; \tau_e)}{\mathcal{L}_{\text{roll}}(\theta_0; \tau_e)}. \tag{4}$$

Here θ_0 denotes the weights of P_0. Scoring both predictors on the same actions controls for trajectory difficulty. A write is rejected when $r_{e,k} > 1.2$ in the rigid-object experiments. Accepted episodes record predictive transfer before replay training. A domain becomes eligible for retrieval when the arithmetic mean of its last three validation ratios is at most 0.9. Write acceptance admits useful experience, while the stricter maturity threshold requires consistent predictive improvement before the learner can guide planning.

During the first three replans, the agent compares eligible predictors and P_0 on the transitions observed so far, $\mathcal{B}_{e,t}$. It selects the predictor with the lowest recent error,

$$k^\star = \underset{k \in \{0\} \cup \mathcal{K}_{\text{eligible}}}{\arg\min} \; \mathcal{L}_{\text{pred}}(\phi_k; \mathcal{B}_{e,t}), \qquad \phi_0 = \theta_0. \tag{5}$$

The selected weights become the base of the fast predictor, preserving the corrections already made in the current episode. For an episode the reference recognizes as familiar, the base stays at P_0. Past trajectories therefore determine retrieval eligibility, while current observations determine which predictor guides planning.

4. Experiments

4.1. Settings

Tasks and backbones. The planner receives an image goal but no shift labels or task-change signals. We study shape and appearance shifts with task-trained ResNet world models [15]. *PushObj* pretraining covers T, L, Z, and plus shapes; deployment also introduces I, square, and small tee. *PushT* keeps the T shape and changes appearance through blur, noise, darkness, or recoloring of the agent, object, or goal. Deformable tasks, *rope* and *granular*, use DINO-WM with pretrained DINOv2 features [20, 26, 42].

Deployment and evaluation. The separate-task PushObj and PushT comparisons use three learned states per planner and five rounds, resampling 50 targets per condition per round. Table 1 reports fifth-round online PushObj results for SE-JEPA. We also evaluate each learned memory on separate test-seed streams with memory writes disabled and retrieval and fast adaptation active. We report success and mean replans, counting the full budget for failures. Rope and granular isolate slow consolidation: one predictor per task revisits 50 fixed instances over four passes, with fast adaptation disabled. We measure final Chamfer distance (CD) to the goal and separately test rope on new instances.

Baselines and planners. We compare with Frozen, AdaJEPA [35], and the predictive and representation baselines in Table 1. Rigid-object planners share a 150-action budget. The paired held-out comparisons evaluate Frozen, AdaJEPA, and SE-JEPA on the same targets. We use released AdaJEPA weights and reproduce its episodic adaptation. Published deformable baseline scores come from Zhou et al. [42]; Appendix B and Appendix C provide evaluation settings and sources.

4.2. Planning with memory on held-out targets

Generalization across shapes and appearances. On shapes absent from pretraining, held-out CEM success is about 15 percentage points higher with SE-JEPA than with AdaJEPA (Table 10). Both planners gain most on I, followed by square and small tee (Fig. 7; Table 11). Stored transitions support planning toward new goals.

On PushT, CEM success under red-object recoloring rises from 14.0% for Frozen and AdaJEPA to 47.1% (Table 12). Replay combines repeated encounters into a reusable predictor, with large gains under red-goal recoloring.

Planning efficiency. Halfway through the GD replan budget, SE-JEPA solves more goals than AdaJEPA reaches with the full budget, on both the seven shapes and seven visual conditions (Fig. 5). Curves include all goals, including failures. CEM gives the same ordering in mean replans for both shapes and visual shifts (Appendix B).

Table 1. **Goal-conditioned planning with CEM.** PushObj covers seven shapes, three unseen in pretraining. The SE-JEPA PushObj cells report fifth-round online performance over three chains; baseline cells use separate benchmark evaluations. PushT covers six visual shifts, including three recolorings. Success rates are percentages. Steps use five-action units, with failures assigned 30 (Appendix B). Appendix C identifies sources for published Chamfer distance (CD) results.

	PushObj			PushT visual shifts			Rope	Granular
Method	All (7) ↑	Unseen ↑	Steps ↓	Shift (6) ↑	Color (3) ↑	Steps ↓	CD ↓	CD ↓
DreamerV3 [12]	7.1	5.3	28.36	5.7	0.0	28.91	2.49	1.05
IRIS [24]	2.9	2.7	29.37	6.0	6.7	28.85	1.11	0.37
R3M [25]	18.9	18.0	25.49	21.7	27.3	25.00	1.13	0.95
DINO CLS [26]	14.9	10.0	26.77	34.7	35.3	22.38	0.84	0.79
DINO-WM [42]	62.0	46.0	16.15	33.0	37.3	22.05	0.491	0.301
LeWM [22]	64.0	50.0	16.28	40.3	16.0	21.45	0.786	0.322
DDP-WM [39]	57.7	43.3	17.59	52.7	57.3	18.39	0.677	0.259
Frozen (pretrained)	43.8	36.2	21.08	48.3	33.6	20.10	0.491	0.301
AdaJEPA [35]	70.4	54.4	16.02	60.2	42.2	18.46	0.629	0.304
SE-JEPA	**80.4**	**73.1**	**12.99**	**76.1**	**65.2**	**14.46**	**0.252**	**0.220**



Figure 3. **Persistent memory resolves planning failures.** Selected GD episodes: unseen I (top) and mixed-stream red object (bottom), with matched starts and goals. Labels count MPC replans; **SE-JEPA** succeeds in six. All runs allow 30 replans of five actions each. Appendix E details the mixed-stream and replay protocols.

On the matched PushT timing set, memory raises peak allocated GPU memory by 0.64 GiB (GD) and 0.44 GiB (CEM) over AdaJEPA, but fewer replans reduce mean episode time by 26.7% and 27.6%, respectively. Appendix A.5 details timing, memory, and consolidation costs.

Consolidation on a second backbone. Under the same 15-push budget, consolidation lowers final CD on rope and granular with fast adaptation disabled (Table 1). Replay thus benefits both task-trained ResNet and pretrained DINOv2 representations.

4.3. Qualitative results

With matched starts and goals, SE-JEPA solves unseen-I and mixed-stream red-object episodes missed by Frozen, DINO-WM, and AdaJEPA (Fig. 3). Slow consolidation also improves rope and granular manipulation (Fig. 4); Appendix E provides complete trajectories.

4.4. Learning over deployment

With targets resampled each round, both GD and CEM improve online success on unseen shapes and recolored scenes from the first to final round (Fig. 6, top). Table 1 reports fifth-round online PushObj results; Appendix B gives paired evaluations after learning.

Revisiting rope and granular reduces CD over successive passes (Fig. 6, bottom). After one rope pass, we froze the predictor and tested 91 new instances. Mean CD fell from pretrained DINO-WM's 0.664 to 0.349, a 47% reduction; the paired bootstrap 95% interval for the mean reduction is [0.162, 0.470] (Appendix B). Both predictors use the same new starts and goals, with weights fixed.

Across seven PushObj rounds, about one quarter of

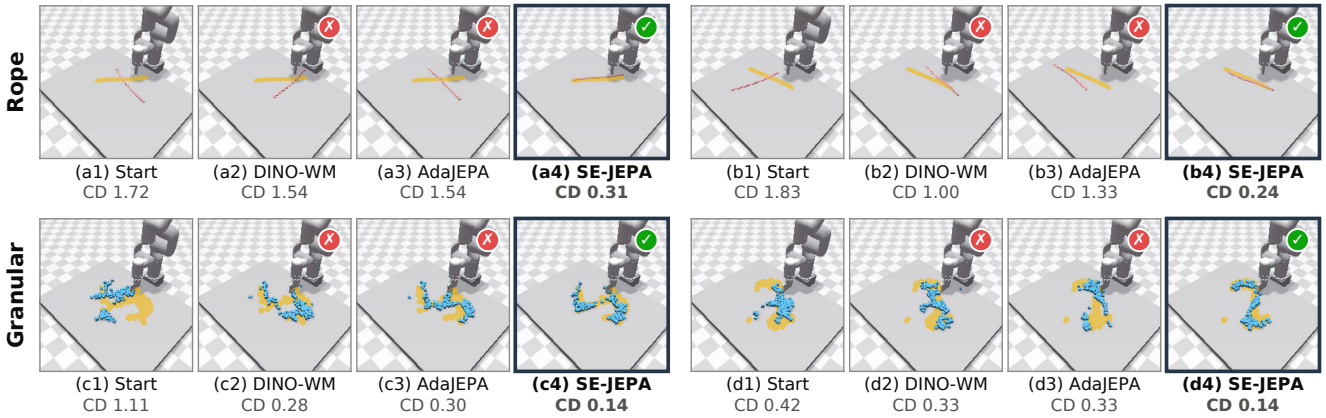


Figure 4. **Consolidated predictions support deformable manipulation.** Real FleX simulator trajectories for rope (top) and granular material (bottom), with two instances per task. Each group shows the start and the three methods' final states after 15 pushes; gold denotes the goal. These third-pass examples illustrate successful slow-only consolidation and were selected where both baselines fail. Frozen DINO-WM frames replay its executed actions. The aggregate fourth-pass results are reported in Table 1.

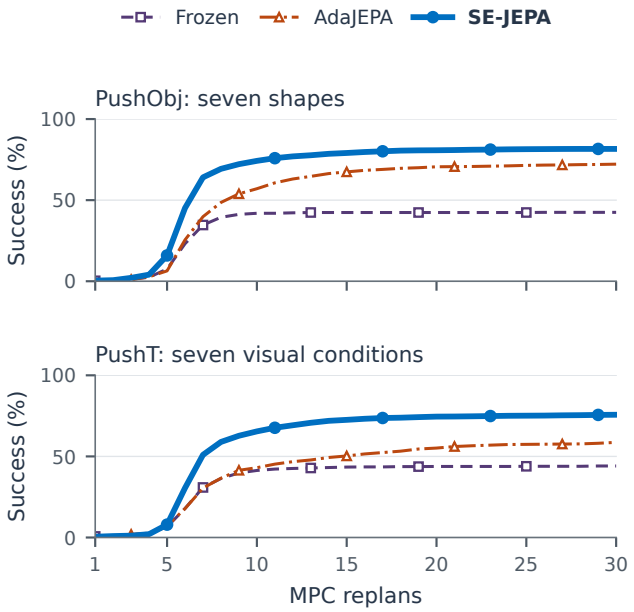


Figure 5. **Higher success within the same interaction budget.** Cumulative held-out success under GD. Curves use all seven shapes (top) and all seven visual conditions (bottom).

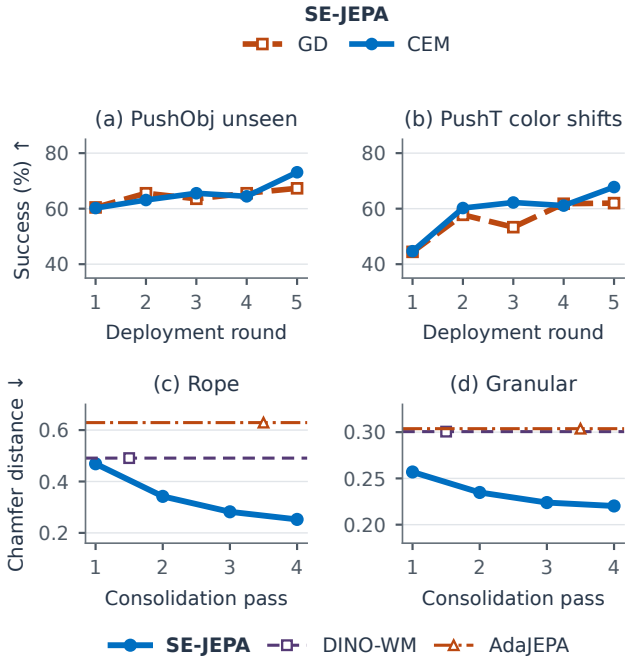


Figure 6. **Deployment experience improves two backbone families.** Top: ResNet-based models, with five rounds of online success on unseen PushObj shapes and three PushT color shifts; curves average three learning chains with targets resampled each round. Bottom: DINO-WM, with four consolidation passes over 50 fixed rope or granular instances; curves show mean CD. The horizontal lines are frozen DINO-WM and AdaJEPA.

episodes contribute to replay, and a similar fraction retrieve a slow predictor at their final replan (Appendix D).

4.5. One memory across task and shift families

We combine seven PushObj shapes and seven PushT visual conditions in one 14-domain stream. Three chains per planner each retain one memory for five rounds, with 50 episodes per domain per round. Targets repeat; adaptation and memory updates remain active.

Averaging GD and CEM equally, online success rises from 65.3% to 75.5% (Fig. 8): shapes improve from 73.4% to 82.0% and visual conditions from 57.2% to 68.9%. One memory thus handles recurring geometric and

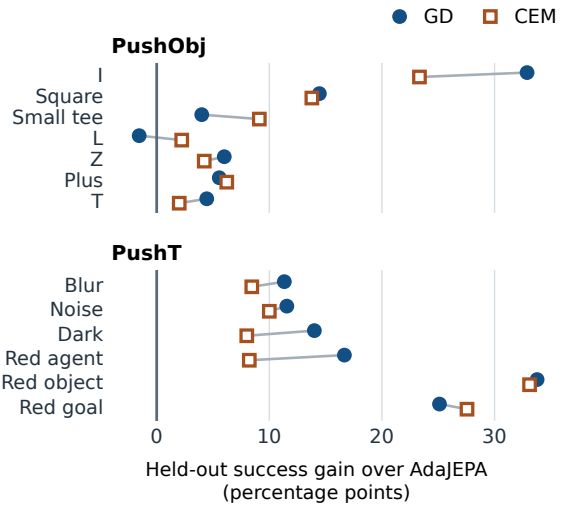


Figure 7. **Held-out success gains over AdaJEPA.** PushObj (top) and PushT (bottom); positive values favor SE-JEPA. I, square, and small tee are unseen during pretraining.

appearance shifts without resets. AdaJEPA and Frozen reach 65.2% and 38.2%, respectively; Appendix D reports both planners and every domain.

Mixed deformable manipulation. We also alternate rope and granular in one CEM stream: three rounds, each with 50 new goals per material, share a rope-trained base and persistent memory. Fast adaptation and an absolute-error write check are enabled (Appendix D.6). Without material labels, SE-JEPA discovers two memories with 100% stored-member purity throughout (Table 2). Slow-predictor retrieval is 98.9% material-correct overall and 100% in the last two rounds. CD decreases for both materials across disjoint trajectory groups, while rope success rises from 60% to 82%. The same memory mechanism therefore separates recurring dynamics in both rigid and deformable manipulation.

Table 2. **Mixed rope/granular learning (CEM).** Purity covers stored members; retrieval accuracy excludes frozen-base selections. Both are percentages; CD is lower-is-better.

Round	Rope CD ↓	Granular CD ↓	Purity	Retrieval
1	0.598	0.471	100	96.6
2	0.452	0.369	100	100.0
3	0.412	0.355	100	100.0

4.6. Ablation study

Accumulating fast updates across episodes. We retain fast updates as $s_{e+1} = s_e + w\Delta_e$, where s_e is the offset from pretrained weights and Δ_e the episode's update. All settings receive the same 100 goals in five blocks (I, square, T, I, square). Resetting reaches 62% success; full inheritance reaches 48%, losing 25 reset successes and recovering 11 (Table 3, exact McNemar $p = 0.029$). The smallest coefficient reaches 64%; retaining larger fractions progressively reduces planning success.

Table 3. **Fast-update accumulation on 100 matched goals.** The coefficient w scales the update retained between episodes. Wins and losses are relative to episodic reset.

Accumulation w	Success (%)	Wins / losses
0 (episodic reset)	62.0	—
0.003	64.0	7 / 5
0.01	59.0	5 / 8
0.03	54.0	4 / 12
1 (naive inheritance)	48.0	11 / 25

Replay rather than parameter averaging. We ablate memory construction under GD and evaluate gates and slow learning rates with both planners (Table 4). Each variant learns for five rounds and is tested on 1,050 held-out goals: seven domains, three seeds, and 50 goals per seed and domain. Within each task, all variants start from the same pretrained model. Appendix D reports success and all-episode replans for each comparison.

Shared replay exceeds fast-parameter EMA by 5.52 points on PushObj and 8.86 on PushT. On PushObj, it solves 44 additional unseen-I goals out of 150. Full SE-JEPA reaches 81.43% and 76.57%, compared with 64.67% and 40.19% for fast-weight inheritance, and exceeds episodic reset by 4.48 and 10.67 points. Retaining transitions supports transfer more effectively than carrying parameter updates forward.

Table 4. **Gate and learning-rate ablations with both planners.** Held-out success (%) over seven domains per task, 1,050 goals per cell. Each task uses one complete learning stream per planner. Appendix D adds replans and the GD memory-design comparisons.

	PushObj		PushT	
Variant	GD	CEM	GD	CEM
Full SE-JEPA	81.43	78.57	76.57	77.90
No maturity screening	80.86	79.24	78.10	75.05
No familiarity protection	83.14	79.62	73.14	75.14
Full SE-JEPA: fixed slow learning rate				
5×10^{-5}	79.62	79.14	73.05	75.81
10^{-4} (default)	81.43	78.57	76.57	77.90
2×10^{-4}	82.38	78.10	76.86	76.76
3×10^{-4}	80.57	77.14	78.10	75.05
5×10^{-4}	78.38	74.38	74.38	73.52

Fixed slow learning rates. Rates from 5×10^{-5} to 3×10^{-4}

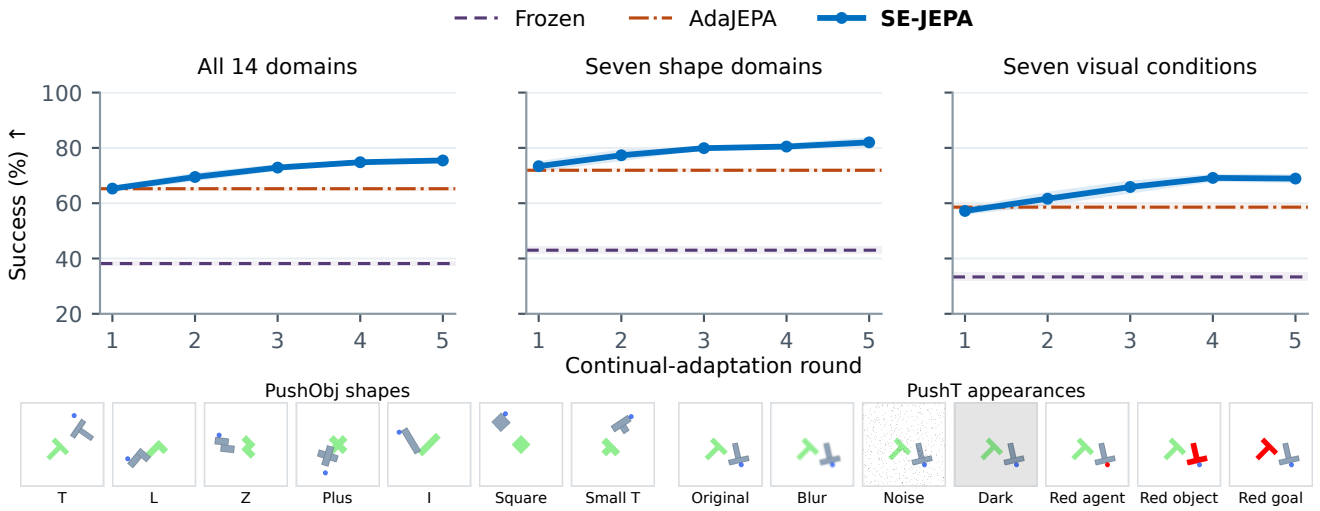


Figure 8. **One memory learns across 14 domains and two task families.** Bottom: recorded observations from each domain. Curves average GD and CEM equally; SE-JEPA retains one memory across five rounds with repeated targets. Frozen and AdaJEPA are archived evaluation references. Shading: sample SD across six runs. Appendix D gives per-domain results.

stay within 1.81 points of the 10^{-4} default on PushObj and 3.52 on PushT. The largest rate, 5×10^{-4}, reduces success in all four comparisons, showing that gains persist across a range of moderate update sizes.

Maturity and familiarity gates. Removing maturity screening changes PushT success from 76.57% to 78.10% with GD but from 77.90% to 75.05% with CEM; PushObj changes by less than one point. Removing familiarity protection reduces PushT success by 3.43 points (GD) and 2.76 (CEM), while improving PushObj by 1.71 and 1.05 points. These planner- and task-dependent effects motivate better calibration of when to store and reuse experience.

Does retrieval select a useful predictor? On common CEM trajectories from 50 new PushT tasks under seven visual conditions, the first three transitions select among the pretrained model and three mature predictors. On subsequent observations, selection reduces one-step MSE by 77.6% versus the pretrained model and 41.1% versus a fixed domain chosen on separate calibration tasks (Table 5). Five-step prediction also improves, demonstrating transfer beyond the selection prefix (Appendix D.11).

The one-step reduction relative to the calibrated single-domain comparator is 0.00909, with paired 95% interval [0.00589, 0.01285]. Among cases eligible for retrieval, selection agrees with the future-best predictor in 82.7% of cases, averaged by condition. Scoring all candidates on common trajectories isolates predictive compatibility: the early observations select dynamics that remain useful later in the encounter.

Table 5. **Selecting mature predictors on new trajectories.** Future latent MSE ($\times 10^{-3}$, lower is better), averaged over seven visual conditions. One-step uses 350 task–condition pairs; five-step uses the 266 with sufficient future observations. The oracle uses future errors for selection.

Predictor selection	One-step ↓	Five-step ↓
Pretrained predictor	58.15	88.94
Fixed domain + pretrained	22.12	47.47
Random eligible predictor	46.74	81.22
Error-based selection (ours)	**13.03**	**31.54**
Future-error oracle	11.79	27.83

5. Conclusion and Future Works

SE-JEPA turns deployment trajectories into predictive memory that improves held-out planning with GD and CEM. One memory handles 14 shape and visual domains; another separates rope from granular while learning both. Replay also generalizes beyond stored experience: a rope predictor learned in one pass reduces goal error on new instances. Prediction uncertainty and planning outcomes could better calibrate the maturity gate. The opposite GD and CEM effects on PushT suggest that readiness should also reflect the planner's use of predictions. Longer observation histories could distinguish similar dynamics, while improved action selection near the goal could turn prediction gains into successful plans. Longer deployments should test how memory growth affects the measured gains in episode time. Comparing compressed memories with full replay buffers would test whether bounded storage can preserve useful experience when rare conditions return.

LLM Usage Statement

ChatGPT was used to assist with language editing and proofreading, including correcting spelling and grammatical errors, improving sentence clarity and fluency, and refining academic writing. It was also used for code debugging. All technical content, experimental results, and scientific claims were reviewed and verified by the authors, who take full responsibility for the final manuscript.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*, 2022. 2

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024. 2

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *IEEE/CVF International Conference on Computer Vision*, 2021. 2

[6] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018. 2

[7] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, 2020. 2

[8] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. 2

[9] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, pages 2555–2565, 2019. 2

[10] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. 2

[11] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations*, 2021.

[12] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. 2, 5

[13] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 2

[14] Nicklas A. Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, pages 8387–8406, 2022. 2

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 4

[16] Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H. Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model-based reinforcement learning for atari. In *International Conference on Learning Representations*, 2020. 2

[17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 2

[18] Yann LeCun. A path towards autonomous machine intelligence. OpenReview position paper, 2022. 1, 2

[19] Zhizhong Li and Derek Hoiem. Learning without forgetting. In *European Conference on Computer Vision*, pages 614–629, 2016. 2

[20] Xingyu Lin, Yufei Wang, Jake Olkin, and David Held. SoftGym: Benchmarking deep reinforcement learning for deformable object manipulation. In *Proceedings of the 2020 Conference on Robot Learning*, pages 432–448, 2021. 4

[21] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017. 2

[22] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. 1, 2, 3, 5

[23] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. 1, 2

[24] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023. 2, 5

[25] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Proceedings of the 6th Conference on Robot Learning*, pages 892–909, 2023. 2, 5

[26] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 2, 4, 5

[27] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, 2019. 2

[28] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016. 2

[29] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. 2

[30] Max Schwarzer, Ankesh Anand, Rishab Goel, R. Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *International Conference on Learning Representations*, 2021. 2

[31] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning*, 2020. 2

[32] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 2

[33] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021. 2

[34] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual test-time domain adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[35] Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. AdaJEPA: An adaptive latent world model. *arXiv preprint arXiv:2606.32026*, 2026. 1, 2, 3, 4, 5, 11

[36] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. In *International Conference on Machine Learning*, 2026. 2

[37] Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, 2015. 2

[38] Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems*, 2021. 2

[39] Shicheng Yin, Kaixuan Yin, Weixing Chen, Yang Liu, Guanbin Li, and Liang Lin. DDP-WM: Disentangled dynamics prediction for efficient world models. *arXiv preprint arXiv:2602.01780*, 2026. 2, 5

[40] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stephane Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *International Conference on Machine Learning*, pages 12310–12320, 2021. 2

[41] Zheyuan Zhang, Suyu Ye, Nakul Agarwal, Hossein Nourkhiz Mahjoub, Ehsan Moradi Pari, Daniel Khashabi, Tianmin Shu, and Vaishnav Tadiparthi. JEPA-TTT: Persistent test-time training of latent world models for planning under dynamics shifts. *arXiv preprint arXiv:2610.00722*, 2026. 1, 2

[42] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, pages 79115–79135, 2025. 1, 2, 3, 4, 5, 14

Supplementary Material

A. Implementation and Memory Management

Compute environment. Model training and evaluation used a server with eight NVIDIA A100 GPUs.

A.1. Fast and slow learning

The label-free rigid-object configuration builds on the episodic adaptation loop of AdaJEPA [35]. At every replan, recent observed transitions provide a self-supervised next-state prediction loss. One Adam step adapts the predictor's last block and the encoder's designated projection/head parameters in working copies, leaving the original pretrained weights fixed. The episode-local predictor and encoder changes are reset at the next episode. A retrieved slow predictor supplies the predictor initialization; it does not replace the frozen reference encoder.

Persistent replay is re-encoded by the reference encoder. Each slow learner is a full predictor copy, initialized from the pretrained predictor rather than from the final fast parameters of the episode that created the domain. For each slow optimization step, up to eight stored episodes are sampled with replacement; each contributes a random window of up to six latent frames. The prediction objective averages one-step observation-feature errors over the available history windows. Actions condition the prediction; the loss is computed only on observation features. Standard writes use 15 Adam steps. Founding a learner after n stored episodes uses $15n$ steps to process the accumulated evidence. Merging likewise adds replay updates for absorbed experience.

Replay stores compact latent trajectories rather than images. The pending pool is bounded, and merging and absorption remove redundant predictor copies as compatible domains are recognized.

Memory and computation. For K domains, persistent model storage consists of K slow predictor copies plus the frozen reference. Replay storage scales with the number of retained latent transitions, rather than the number of pixels in the original observations. Retrieval scores eligible predictors during the first three replans; slow optimization occurs after an accepted write, with 15 updates per standard write. Thus interaction efficiency and computation depend on different quantities: the former on replans until success, the latter on candidate count, retained experience, and replay updates. Domain merging shares both stored experience and a single surviving learner.

A.2. Order of decisions

Algorithm 1 summarizes a learning episode. Familiarity uses the geometric mean of newest-frame prediction errors on up to the first three observed transitions. The threshold is calibrated once on the shape backbone and transfers unchanged to the visual experiments.

The selected predictor's error is evaluated before it learns from the current episode, so maturity reflects transfer to data the predictor has not fitted. In read-only evaluation, the episode-end write, validation-history update, slow training, merging, and absorption steps are all disabled.

Table 6. Label-free configuration used for PushObj and PushT. The seven-round stream in Appendix D uses a slow learning rate of 2×10^{-4} for all replay updates.

Setting	Value
Fast predictor learning rate	5×10^{-4}
Fast encoder-head learning rate	10^{-5}
Fast steps per replan	1
Recent adaptation buffer	5 transitions
Slow learning rate	10^{-4}
Slow steps per standard write	15
Slow batch size (episodes)	8
Slow sampled window (frames)	up to 6
Familiarity threshold τ_f	0.0146
Existing-domain cosine threshold	0.10
New-domain cosine threshold	0.05
Episodes to confirm a domain	3
Pending-pool capacity	40
Minimum episodes to train a learner	3
Retrieval window	first 3 replans
Maturity history length m	3
Maturity error-ratio threshold τ_m	0.9
Write rejection ratio	1.2
Merge evidence per direction	2 episodes
Merge relative tolerance	0.5
Fragment absorption interval	25 episodes
Absorption error-ratio threshold	0.8
Fragment-size limit	< 6 episodes

A.3. Fingerprints and delayed domain founding

A fingerprint concatenates the adapted predictor and encoder-head changes and is normalized to unit length. Domain prototypes accumulate normalized fingerprints from episodes planned from the reference; episodes starting from a slow predictor do not redefine that reference-based prototype. If an episode retrieved a slow predictor, its trajectory is first considered for that predictor's domain, subject to the error-ratio check before replay. Otherwise, it tries compatible domain directions in descending cosine order.

An unassigned episode enters a pending pool. It confirms a new domain when at least two pending fingerprints align with its own above the new-domain threshold. These members need not be pairwise aligned with one another; the incoming fingerprint is the common reference for this

Algorithm 1 One deployment-learning episode of SE-JEPA

Require: Frozen (E_0, P_0), memory $\mathcal{M}$, initial observation o, goal g
Reset fast parameters to (E_0, P_0)
Initialize recent buffer $\mathcal{B}$ and episode trajectory τ
while budget remains and task is not complete **do**
    Optimize an action sequence with the current fast model
    Execute its first block; observe the next state
    Append real transitions to $\mathcal{B}$ and τ
    if within the early retrieval window **then**
        Form candidates: P_0 and mature slow predictors
        Restrict to P_0 if the observed episode is familiar
        Select the smallest observed prediction error
        Load the selected predictor, retaining the fast offset
    end if
    Adapt the fast model on recent real transitions
end while
Encode τ with frozen E_0
Measure pre-update rollout errors of existing predictors
if episode is unfamiliar **then**
    Propose a domain using retrieval or an update fingerprint
    Reject an incompatible write, or retain pending evidence
    if a domain is assigned or newly confirmed **then**
        Record validation before replay training when a learner exists
        Append latent experience; update its slow predictor
        Check bidirectional merges and periodic absorption
    end if
end if
Discard episode-local fast parameters

confirmation. The members and the new episode are transferred together to the new replay. No ground-truth shape or appearance category participates in this procedure.

A.4. Merging and fragment absorption

For a new unfamiliar episode, every existing trained learner is evaluated before training. The resulting ratios provide cross-domain evidence. Let $G_{A\to B}$ be the geometric mean error ratio of learner B on recent episodes assigned to A, and $G_{A\to A}$ the corresponding ratio for learner A. Domains merge only if both directions satisfy

$$G_{A\to B} \le \min\{1, (1+\epsilon_m)G_{A\to A}\}, \tag{6}$$

with the symmetric condition for $B \to A$. Each direction requires two jointly evaluated episodes in the primary configuration. The larger domain retains its learner and incorporates the union of stored experience. Identifiers of merged domains remain retired for the rest of the stream.

Every 25 episodes, small immature domains can be absorbed into a mature predictor that explains their experience with an error ratio at most 0.8. The primary configuration limits absorption to fragments with fewer than six episodes and fewer than three maturity measurements. Pending-pool absorption and the additional absolute-error gate are disabled in the shape and visual experiments.

A.5. Measured deployment cost

We profile Frozen, AdaJEPA, and SE-JEPA on one NVIDIA RTX 4090 GPU per run, using the same ten physical PushT tasks under seven visual conditions. Each method–planner pair has 70 task–condition pairs and three timing repetitions, giving 1,260 measured episodes overall. Two warm-up episodes precede each run. We average repetitions within each pair before computing mean and 95th-percentile (P95) episode times. CUDA operations are synchronized for timing. All methods use the same planner implementation within each planner family; GD builds its CUDA graphs during warm-up.

Time and GPU memory. The measurement evaluates the memory learned after five rounds, with retrieval and fast adaptation active and persistent memory fixed. Episode time includes planning, adaptation, retrieval, simulator execution, and evaluation, but excludes process initialization and warm-up. Table 7 includes successful and unsuccessful episodes. Relative to AdaJEPA, SE-JEPA reduces mean replans from 23.56 to 15.91 under GD and from 23.09 to 15.69 under CEM, lowering mean episode time from 18.30 to 13.42 seconds and from 21.35 to 15.45 seconds. The respective speedups are $1.36\times$ and $1.38\times$. Peak allocated GPU memory increases by 0.64 and 0.44 GiB. Allocated memory measures live tensors; reserved memory also includes the allocator cache.

Where the time is spent. Table 8 separates the major episode costs. Planning itself costs about 0.30 seconds per replan with GD and 0.44–0.45 seconds with CEM for all methods. The reduction in total time comes from completing more tasks with fewer replans. Adaptation includes prediction diagnostics, and retrieval adds only 0.094 seconds per GD episode and 0.118 seconds per CEM episode on average. P95 episode time is higher for SE-JEPA: difficult episodes can still exhaust the budget while paying the additional adaptation and retrieval costs.

Table 7. **Deployment time and GPU memory on matched PushT tasks.** Success (%), mean replans, mean and P95 episode time (seconds), and maximum allocated/reserved GPU memory (GiB). Timing repetitions share the same tasks and learned memory.

Planner	Method	Success ↑	Replans ↓	Mean time ↓	P95 time	Allocated	Reserved
GD	Frozen	21.43	24.94	17.88	23.64	0.21	0.30
GD	AdaJEPA	32.86	23.56	18.30	25.52	0.30	0.36
GD	SE-JEPA	67.14	15.91	13.42	28.62	0.93	1.02
CEM	Frozen	22.86	24.71	21.26	27.56	2.66	3.00
CEM	AdaJEPA	35.71	23.09	21.35	29.96	2.69	3.01
CEM	SE-JEPA	71.43	15.69	15.45	33.06	3.13	3.91

Table 8. **Mean episode time by component (seconds).** Adaptation includes diagnostics; simulator time includes evaluation. Minor reset and bookkeeping costs account for the remaining difference from Table 7.

Planner	Method	Planning	Adaptation	Retrieval	Simulator/evaluation
GD	Frozen	7.568	0.000	0.000	10.267
GD	AdaJEPA	7.140	1.722	0.000	9.401
GD	SE-JEPA	4.782	3.188	0.094	5.329
CEM	Frozen	11.007	0.000	0.000	10.214
CEM	AdaJEPA	10.355	1.730	0.000	9.226
CEM	SE-JEPA	6.905	3.237	0.118	5.164

Learning and persistent storage. Table 9 reports the preceding five-round learning stage, with 1,750 episodes per stream. Each final memory holds 733–766 trajectories in the profiled streams; their latent replay tensors occupy 21.5–22.9 MiB. The full saved state also contains predictor and optimizer tensors, fingerprints, and pending experience. Across all six streams, consolidation uses 11,280–15,180 slow optimizer steps, including learner initialization, ordinary writes, merges, and absorption. These counts are reconstructed from the executed update rules and recorded events and reconciled with the saved replay states. The deployment timings above measure reuse of these learned memories; earlier consolidation is reported separately.

B. Evaluation Protocols

B.1. Shape shifts

The shape learning stream has five deployment rounds over seven shapes, with 50 episodes per shape in every round. The post-learning benchmark uses target seeds 2000, 2100, and 2200, each with 350 episodes. I, square, and small tee are unseen in pretraining; T, L, Z, and plus are seen.

For the paired planner comparison in Table 10, each planner has three learned memory states. Every state is evaluated on all three test seeds. For each seed we average over episodes and states, then compute the standard deviation across seed-level means. The local external PushObj baselines use seed 2000.

Table 1 reports fifth-round online PushObj performance with adaptation and memory writing active: 844 successes out of 1,050 episodes (80.38%) overall, and 329 out of 450 (73.11%) on unseen shapes. Figure 1 reports the gains over AdaJEPA on held-out targets after learning, for both GD and CEM.

B.2. Visual shifts

The paired visual evaluation contains the original T condition, blur, salt-and-pepper noise, darkness, red agent, red object, and red goal. Three five-round learning chains resample targets every round, with 350 episodes per round. Held-out testing uses three target seeds and 50 episodes per condition. Direct checks of source-episode identifiers confirm that held-out sources never occur in the learning streams and that the three test seeds use disjoint sources.

The color-learning curve in Fig. 6 aggregates red-agent, red-object, and red-goal episodes. For each round and learning chain, it sums successes over the three conditions and divides by 150 episodes, and plots the mean of the three chain-level rates. The archived statistics include their sample standard deviation. The three conditions share the same 50 source targets within a chain and round.

The external visual baselines use the established VIS50 collection with planning seed 100. The external comparisons use its six shifted conditions, with 50 paired initial/goal states per condition.

B.3. Success and interaction counts

For the paired rigid-object comparisons, GD uses 100 optimization steps; CEM uses 10 iterations with 200 samples and 30 elites per iteration. Rigid-object success requires a

Table 9. **Memory after five learning rounds.** Active/mature predictor counts, retained trajectories, latent replay and full serialized state in MiB, and cumulative slow optimizer steps. Storage on disk is distinct from peak runtime GPU allocation.

Planner	Stream	Active/mature	Trajectories	Replay MiB	State MiB	Slow steps
GD	1	4 / 1	766	22.9	525.4	13,455
GD	2	2 / 1	747	22.5	330.4	13,155
GD	3	5 / 2	742	20.5	576.0	15,180
CEM	1	3 / 3	733	21.5	454.8	11,280
CEM	2	5 / 3	750	21.8	577.9	13,080
CEM	3	4 / 2	740	20.8	522.6	12,960

joint Euclidean error below 20 simulator units across the pusher and object positions, and an object orientation error below $\pi/9$. Each executed model action contains five physical actions, and an episode has at most 30 MPC replans. For episode i with success indicator S_i and first successful replan T_i,

$$\overline{T} = \frac{1}{N}\sum_{i=1}^{N}\left[\, S_i T_i + (1-S_i)\, T_{\max}\right], \qquad T_{\max} = 30. \quad (7)$$

Equivalently, $\overline{T} = T_{\max} - \sum_{t=1}^{T_{\max}-1} F(t)$, where $F(t)$ is the fraction of episodes solved by replan t; the cumulative curves in Fig. 5 visualize F.

DINO-WM PushT CEM executes 25 actions per replan; Table 1 converts its mean to five-action units, $4.41 \times 25/5 = 22.05$, with failures assigned 30 units.

B.4. Deformable manipulation

Rope and granular each use one slow predictor, a DINO-WM backbone, and 50 instances revisited over four passes, with the same targets, order, and planning seeds in every pass. The slow-only variant plans from the slow predictor without fast within-episode updates. CEM uses 300 samples, 30 elites, and 10 iterations, with three MPC iterations of five pushes each. The unseen-rope evaluation freezes the predictor saved after the first pass and tests it on 91 instances absent from the learning stream, against the frozen reference on the same instances. Mean CD falls from 0.664 with the reference to 0.349 with the one-pass predictor. Across the 91 matched instances, the predictor improves CD on 56. A paired percentile bootstrap over instances (20,000 resamples, seed 20261107) gives a 95% interval of [0.162, 0.470] for the mean CD reduction. The observed mean reduction is 0.314.

C. Baseline Implementations and Sources

Model initialization. Frozen and AdaJEPA use the official task-specific checkpoints on PushObj and PushT. SE-JEPA uses task-trained ResNet world models; its PushObj backbone is trained for three epochs on 16,000 T, L, Z, and plus trajectories. The additional DINO-WM shape model uses the same trajectory count and epoch budget. Its updated PushT CEM checkpoint uses 16,000 newly generated random T-shape trajectories and epoch three. DreamerV3, IRIS, DINO CLS, and R3M use T-only checkpoints. DreamerV3 and IRIS are frozen offline world-model adapters for continuous-action CEM planning, without reward or policy learning. Configurations and source records are archived with the paper.

Evaluation groups. Frozen, AdaJEPA, and SE-JEPA use the paired held-out targets in Appendix B. External visual baselines use VIS50: 50 initial/goal pairs per condition and seed 100. PushT Shift averages blur, noise, dark, red agent, red object, and red goal; Color averages the last three. The original condition enters the seven-condition curves. DreamerV3 and IRIS PushObj results use 50 targets per shape at seed 2000. The Unseen column consistently averages I, square, and small tee for all methods. Methods within each group share targets and the total action budget.

Published results and deformable references. The CDs for DreamerV3, IRIS, R3M, and DINO CLS are reported by Zhou et al. [42] on ten instances per task. Our DINO-WM and Frozen deformable rows share the same fixed reference on 50 instances. Appendix B describes consolidation and the separate evaluation on unseen rope instances.

D. Additional Results

D.1. All object shapes

Table 11 reports every held-out shape. SE-JEPA gives the largest gains on the unseen shapes: with GD, I rises from 44.7% (AdaJEPA) to 77.6% and square from 46.7% to 61.1%. It also leads on the seen shapes in aggregate.

D.2. All visual conditions

Table 12 and Fig. 7 break the paired visual results down by condition. SE-JEPA improves on all seven conditions with both planners. The largest gains, more than 25 points, appear under red-object and red-goal recoloring, where

Table 10. Both planners on the paired post-learning evaluation sets. Shape All and Unseen use the separate PushObj test-seed streams. Visual All averages seven conditions and Shift six; visual Steps averages all seven conditions.

		PushObj			PushT		
Planner	Method	All ↑	Unseen ↑	Steps ↓	All ↑	Shift ↑	Steps ↓
GD	Frozen	42.48	32.44	20.02	44.10	41.56	19.94
GD	AdaJEPA	72.19	55.33	14.52	58.67	54.89	17.96
GD	SE-JEPA	81.59	72.44	11.30	75.68	73.63	13.21
CEM	Frozen	43.81	36.22	21.08	50.86	48.33	19.56
CEM	AdaJEPA	70.38	54.44	16.02	62.95	60.22	17.87
CEM	SE-JEPA	79.08	69.85	13.53	77.05	76.11	14.15

Table 11. Held-out success (%) per PushObj shape. I, square, and small tee are unseen in pretraining. All averages the seven shapes.

Planner	Method	I	L	Square	Z	Small tee	Plus	T	All
CEM	Frozen	32.7	52.7	23.3	41.3	52.7	43.3	60.7	43.8
CEM	AdaJEPA	49.3	82.0	44.7	82.0	69.3	79.3	86.0	70.4
CEM	SE-JEPA	72.7	84.2	58.4	86.2	78.4	85.6	88.0	79.1
GD	Frozen	24.0	52.7	28.0	40.0	45.3	48.7	58.7	42.5
GD	AdaJEPA	44.7	90.0	46.7	82.0	74.7	82.0	85.3	72.2
GD	SE-JEPA	77.6	88.4	61.1	88.0	78.7	87.6	89.8	81.6

episodic adaptation has to learn the same appearance correction again for each new target.

D.3. One memory across fourteen domains

This experiment places seven PushObj shapes and seven PushT visual conditions in one continual stream. The 14 domains form two recurring families, rather than all shape–appearance combinations. Each round presents all seven visual conditions followed by all seven shapes, with the shape order alternating between rounds. One memory state carries through every domain and round. Three chains per planner start with an empty memory and run for five rounds, with 50 episodes per domain per round. The same initial/goal pairs recur throughout; the changing seeds control run randomness. The curves measure continued learning on recurring goals; Appendix B evaluates held-out planning.

Improvement across both families. Section 4.5 and Figure 8 report five-round success averaged equally over GD and CEM. The figure's horizontal baselines average the archived evaluations, not additional five-round trajectories; shading is the sample SD of six planner-by-chain or planner-by-seed success rates. Each online round totals 4,200 episodes across both planners. The thumbnail strip shows the recorded initial observation of episode zero in each domain, from GD chain two in round five. Selection uses a fixed episode index independent of success; the native images retain the actual appearance corruptions. Both families improve from the first to fifth round: GD rises from 56.8% to 69.1% on visual conditions and from 74.4% to 81.1% on shapes; CEM rises from 57.6% to 68.7% and from 72.4% to 82.9%, respectively, with one memory retained throughout.

Final-round planning outcomes. Table 13 compares the fifth round of SE-JEPA with the archived Frozen and AdaJEPA baselines. SE-JEPA reaches 75.1% (GD) and 75.8% (CEM), with higher success in both families and fewer replans overall. Relative to AdaJEPA, the observed overall gaps are 10.0 and 10.4 percentage points, while all-episode replans decrease by 18.8% and 17.6%. These gains combine a higher fraction of solved goals with a lower interaction cost. Table 14 reports every domain: SE-JEPA exceeds AdaJEPA in 12 of 14 domains with GD, and in 13 with CEM, tying on noise.

Frozen retains the pretrained predictor, AdaJEPA adapts within each episode, and SE-JEPA retains predictive memory across the stream. The matched initial/goal pairs let us compare planning outcomes as geometric and visual conditions recur.

D.4. Planning efficiency

Figure 5 plots cumulative held-out success against the number of MPC replans. SE-JEPA pulls ahead within the first four replans and keeps the lead for the rest of the budget, showing how the advantage develops before the full interaction budget of 30 MPC replans is exhausted.

Table 12. Held-out visual success (%) per condition on the paired targets. All averages seven conditions; Shift excludes Original.

Planner	Method	Original	Blur	Noise	Dark	Red agent	Red object	Red goal	All	Shift
CEM	Frozen	66.0	56.0	62.7	70.7	50.0	14.0	36.7	50.9	48.3
CEM	AdaJEPA	79.3	78.0	76.7	80.0	71.3	14.0	41.3	63.0	60.2
CEM	SE-JEPA	82.7	86.4	86.7	88.0	79.6	47.1	68.9	77.0	76.1
GD	Frozen	59.3	48.7	55.3	58.7	50.7	6.7	29.3	44.1	41.6
GD	AdaJEPA	81.3	77.3	73.3	73.3	58.0	10.7	36.7	58.7	54.9
GD	SE-JEPA	88.0	88.7	84.9	87.3	74.7	44.4	61.8	75.7	73.6

Table 13. **Mixed 14-domain online results.** Success (%) and all-episode MPC replans, mean $\pm$ sample SD across three chains (SE-JEPA, round five) or three seeds (baselines). Each row has 2,100 episodes, split equally between the two families. Failures use the full budget.

Planner	Method	All success ↑	Shape success ↑	Visual success ↑	Replans ↓
GD	Frozen	37.7 ± 0.0	42.0 ± 0.0	33.4 ± 0.0	21.29 ± 0.00
GD	AdaJEPA	65.1 ± 0.7	72.0 ± 0.9	58.2 ± 0.7	16.04 ± 0.07
GD	**SE-JEPA**	$\mathbf{75.1 \pm 1.4}$	$\mathbf{81.1 \pm 2.3}$	$\mathbf{69.1 \pm 1.6}$	$\mathbf{13.03 \pm 0.32}$
CEM	Frozen	38.6 ± 0.9	44.0 ± 1.0	33.2 ± 2.5	21.93 ± 0.33
CEM	AdaJEPA	65.4 ± 0.7	71.8 ± 0.4	59.0 ± 1.4	17.28 ± 0.25
CEM	**SE-JEPA**	$\mathbf{75.8 \pm 0.8}$	$\mathbf{82.9 \pm 1.0}$	$\mathbf{68.7 \pm 1.9}$	$\mathbf{14.24 \pm 0.18}$

D.5. Deformable manipulation

Figure 6 shows the four-pass learning curves in the main paper. Table 15 adds standard errors to the deformable results of Table 1. Table 16 follows consolidation pass by pass at two slow learning rates. With 10^{-4}, CD decreases at every pass on both tasks. Rope improves faster in the early passes at 2×10^{-4}; the two learning rates yield similar final goal errors after four passes on both deformable tasks.

D.6. Mixed rope and granular learning

We also test whether one persistent memory can separate and learn two deformable dynamics regimes in the simulated robot-arm environment. A single CEM chain alternates ten rope episodes with ten granular episodes, five times per round, for three rounds and 300 goals. Each round uses 50 goals per material, with no source trajectory shared across rounds. Both materials use the same rope-trained DINO-WM initialization, so granular is a new material for the pretrained predictor. Memory starts empty and persists across every material switch. Material labels are used only for analysis; the algorithm receives neither domain identities nor a prescribed number of domains.

This mixed-stream configuration retains fast adaptation and uses a slow learning rate of 2×10^{-4}. It also checks that an episode's prediction error is at most 20 times the candidate predictor's median error on its own stored trajectories before accepting a write. This absolute-error check supplements the relative validation test in Appendix A. Each episode allows 15 pushes, and success requires final CD below 0.5 for rope or 0.2 for granular. These runs share one base and retain fast adaptation; the single-domain results use separate bases without fast adaptation.

Table 17 shows that the system discovers two mature memories and keeps them distinct: after three rounds, all 80 stored rope episodes and all 150 granular episodes belong to their respective material domains. The other 70 episodes are 69 familiar rope encounters and one rejected write. Stored-member purity is 100% at every round endpoint. Of 660 slow-predictor selections, 653 (98.9%) match the current material; all seven mismatches occur in the first round. All 457 slow-predictor selections in rounds two and three are material-matched, while the frozen reference remains available throughout deployment.

Mean final CD falls from 0.598 to 0.412 on rope and from 0.471 to 0.355 on granular across the three disjoint trajectory groups. Rope success rises from 60% to 82%; granular success is 2%, 8%, and 4%. Together with material-correct retrieval, the decreasing goal distances show that one label-free memory can retain distinct dynamics and continue learning as materials alternate.

D.7. Component protocols and learning rates

Tables 18 and 19 use one five-round learning stream per task and planner. The final memory is tested with writes disabled on seeds 2000, 2100, and 2200, with 50 goals per domain and seed. Each cell summarizes 1,050 evaluations over seven domains. PushT includes the original appearance and six shifts. Replans average over all episodes, counting failures as 30; all ablations within a task share the pretrained initialization.

Table 14. **Mixed-stream success (%) for every domain.** Each entry averages 150 episodes under the protocol of Table 13. Original appearance and the T-shape block use different target sets and are retained as separate domains.

		GD			CEM		
Family	Domain	Frozen	AdaJEPA	SE-JEPA	Frozen	AdaJEPA	SE-JEPA
Visual	Original	50.0	70.7	**78.7**	46.7	72.0	**78.0**
Visual	Blur	32.0	72.7	**78.0**	30.7	70.0	**72.7**
Visual	Noise	44.0	76.7	**78.7**	41.3	**73.3**	**73.3**
Visual	Dark	48.0	72.0	**76.0**	48.7	73.3	**74.7**
Visual	Red agent	36.0	56.0	**62.0**	38.0	56.7	**71.3**
Visual	Red object	6.0	13.3	**52.0**	6.0	21.3	**50.7**
Visual	Red goal	18.0	46.0	**58.7**	21.3	46.0	**60.0**
Shape	I	40.0	56.7	**82.7**	26.0	54.0	**88.0**
Shape	L	52.0	**83.3**	82.0	49.3	82.7	**90.0**
Shape	Square	14.0	41.3	**60.0**	26.7	38.7	**59.3**
Shape	Z	34.0	78.7	**79.3**	42.7	78.7	**83.3**
Shape	Small tee	52.0	**78.7**	76.0	56.7	80.0	**81.3**
Shape	Plus	54.0	84.7	**94.0**	46.7	81.3	**86.7**
Shape	T	48.0	80.7	**94.0**	60.0	87.3	**91.3**

Table 15. Deformable manipulation: mean CD $\pm$ standard error over 50 instances. SE-JEPA uses its slow predictor after four passes through the same 50 instances.

Method	Rope CD ↓	Granular CD ↓
DINO-WM	0.491 ± 0.059	0.301 ± 0.020
LeWM	0.786 ± 0.112	0.322 ± 0.018
DDP-WM	0.677 ± 0.086	0.259 ± 0.012
AdaJEPA	0.629 ± 0.096	0.304 ± 0.019
SE-JEPA	$\mathbf{0.252 \pm 0.045}$	$\mathbf{0.220 \pm 0.012}$

Table 16. CD of SE-JEPA after each consolidation pass at two slow learning rates (mean $\pm$ standard error, 50 instances).

	lr 10^{-4}		lr 2×10^{-4}	
Pass	Rope	Granular	Rope	Granular
1	0.469 ± 0.100	0.257 ± 0.014	0.416 ± 0.070	0.260 ± 0.012
2	0.342 ± 0.054	0.235 ± 0.012	0.286 ± 0.050	0.241 ± 0.012
3	0.282 ± 0.033	0.224 ± 0.010	0.283 ± 0.037	0.247 ± 0.012
4	0.252 ± 0.045	0.220 ± 0.012	0.243 ± 0.033	0.243 ± 0.013

Memory design. The episodic-reset variant removes persistent memory. Direct inheritance carries fast weights to the next goal. The shared-replay variant trains one slow predictor on stored transitions; EMA instead averages fast parameters with mixing coefficient $\alpha = 10^{-4}$. Table 18 isolates these choices under GD. Replay exceeds EMA on both tasks, and domain-specific replay gives the highest success among these designs. Relative to direct inheritance, full SE-JEPA reduces mean replans from 16.12 to 11.37 on PushObj and from 22.33 to 12.93 on PushT.

Planner consistency. Table 19 evaluates the complete gate and learning-rate grid with both GD and CEM. Removing maturity screening changes which slow predictors can compete for retrieval, while preserving prediction-error selection. Removing familiarity protection allows familiar episodes to retrieve slow predictors and enter replay. The GD-only rows in Table 18 also separate the read and write protections. The two mechanisms act at different stages: familiarity controls participation in memory, whereas maturity controls predictor readiness.

With either planner, removing familiarity protection improves PushObj but reduces PushT success. The maturity effect also depends on the planner: earlier retrieval benefits PushT GD, whereas screening benefits PushT CEM. A single fixed gate therefore offers a useful operating point across settings, with further gains possible through task-aware calibration that does not require domain labels. Across the learning-rate sweep, 5×10^{-4} consistently performs worse than the default; the smaller rates preserve strong results with both optimizers.

D.8. Transferring fast weights across goals

We tested whether the parameter update from one episode helps on the next. Let s_e be the offset from the pretrained weights at the start of episode e, and Δ_e the fast update in that episode. We set $s_{e+1} = s_e + w\Delta_e$, where $w = 0$ resets to the pretrained model and $w = 1$ retains the full update. Each variant sees the same 100 PushObj goals in five blocks of 20 (I, square, T, I, square). Keeping the full update loses more goals than it gains relative to reset (Table 3, two-sided exact McNemar $p = 0.029$). Smaller weights give success rates closer to those of independent episodic reset.

Table 17. **Mixed deformable learning and domain separation.** One CEM chain, 50 new goal segments per material and round. Stored R/G counts are cumulative rope/granular members. Purity measures the fraction of stored members matching their domain's majority material. Correct retrievals count material-matched slow-predictor selections at MPC replans; selections of the frozen reference are excluded.

	Rope		Granular		Memory and retrieval		
Round	CD ↓	Success (%) ↑	CD ↓	Success (%) ↑	Stored R/G	Purity (%)	Correct retrievals
1	0.598	60	0.471	2	27 / 50	100	196/203
2	0.452	74	0.369	8	50 / 100	100	219/219
3	0.412	82	0.355	4	80 / 150	100	238/238

Table 18. **Memory-design ablations (GD).** Each entry reports success (%) / all-episode MPC replans on 1,050 held-out goals. Variants share five learning rounds and paired test goals.

Variant	PushObj	PushT
Full SE-JEPA	81.43 / 11.37	76.57 / 12.93
Episodic reset	76.95 / 12.54	65.90 / 15.47
Inherit fast weights	64.67 / 16.12	40.19 / 22.33
Shared predictor: EMA	75.71 / 12.84	66.29 / 15.30
Shared predictor: replay	81.24 / 11.41	75.14 / 13.26
No familiar-read protection	76.95 / 12.54	76.19 / 13.16
No familiar-write protection	79.43 / 11.76	73.81 / 13.66

D.9. A seven-round shape stream

A second PushObj stream runs seven rounds with a slow learning rate of 2×10^{-4}, one chain per planner. Each round has 50 episodes per shape, drawn without replacement and excluding the held-out sources, for 2,450 episodes per planner. Online success rises from 70.6% in the first round to 78.6% in the seventh with GD, and from 72.3% to 78.3% with the sampling-based CEM planner.

The episode logs record the fast-update fingerprint of every episode. Figure 9 shows the cosine between consecutive fingerprints: the mean is 0.018 for both planners. Absolute cosine is below 0.1 for 96.3% of GD pairs and 96.8% of CEM pairs. These are descriptive statistics from one sequential stream per planner. These measurements support consolidating transition data rather than accumulating episode-specific parameter updates (Sec. 3.1).

D.10. Memory use across episodes

SE-JEPA learns selectively. In a seven-round PushObj stream of 2,450 episodes per planner, it writes only 25% (GD) to 27% (CEM) of episodes to memory; the rest are familiar to the pretrained model, still pending, or rejected by validation. About one quarter of episodes retrieve a slow predictor; the others adapt from the pretrained model.

Selective writing leaves familiar dynamics to the pretrained model, whose original weights remain fixed. On the four pretraining shapes, SE-JEPA reaches 86.0% held-out success with CEM and 88.4% with GD, against 82.3% and 84.8% for AdaJEPA. Separate slow predictors learn recurring changes from replayed trajectories.

D.11. Predictive compatibility of mature domains

Table 5 evaluates the three mature predictors retained after five CEM learning rounds on PushT. We select 50 new physical start/goal pairs, each rendered under seven visual conditions, and a disjoint set of ten calibration pairs. These pairs exclude the trajectories used in the preceding learning and evaluation streams. The pretrained predictor collects a common CEM trajectory for each pair; all candidate predictors are then scored on the same observed states and actions with fast and slow learning disabled.

Selection precedes evaluation. The deployed rule scores the pretrained reference and eligible mature predictors on the first three observed transitions. Familiarity protection retains the reference in 195 of the 350 task–condition pairs; the remaining 155 admit all four candidates. A fixed-domain comparator retains only the reference and one mature predictor, chosen using the 70 calibration evaluations and locked before testing. Random selection averages uniformly over the eligible candidates. The future-error oracle is the minimum error among all four candidates over the subsequent observations.

One-step MSE averages prediction errors strictly after the selection prefix. Five-step MSE uses autoregressive prediction with the recorded actions and requires a full five-step future window; 266 pairs meet this requirement. Every method uses the same valid pairs for each metric, and the reported aggregate weights the seven conditions equally. This diagnostic isolates predictive compatibility on common observed trajectories.

Evidence for useful retrieval. Selected predictors achieve one-step MSE 0.01303, compared with 0.05815 for the pretrained model, 0.02212 for the calibrated fixed-domain comparator, and 0.04674 for random eligible selection. The oracle reaches 0.01179. The mean reduction from the fixed-domain comparator is 0.00909, with paired 95% interval [0.00589, 0.01285]; the reduction from random selection is 0.03371 [0.02847, 0.03932].

Table 19. **Gate and slow learning-rate ablations (GD and CEM).** Each entry is success (%) / mean MPC replans over 1,050 held-out goals. The final state of one complete five-round learning stream is evaluated for each task and planner.

	PushObj		PushT	
Variant	GD	CEM	GD	CEM
Full SE-JEPA	81.43 / 11.37	78.57 / 13.64	76.57 / 12.93	77.90 / 14.02
No maturity screening	80.86 / 11.46	79.24 / 13.51	78.10 / 12.63	75.05 / 14.58
No familiarity protection	83.14 / 10.90	79.62 / 13.12	73.14 / 13.47	75.14 / 14.18
Full SE-JEPA: fixed slow learning rate				
5×10^{-5}	79.62 / 11.79	79.14 / 13.56	73.05 / 13.65	75.81 / 14.31
10^{-4} (default)	81.43 / 11.37	78.57 / 13.64	76.57 / 12.93	77.90 / 14.02
2×10^{-4}	82.38 / 11.26	78.10 / 13.89	76.86 / 12.86	76.76 / 14.33
3×10^{-4}	80.57 / 11.72	77.14 / 14.01	78.10 / 12.78	75.05 / 14.45
5×10^{-4}	78.38 / 12.40	74.38 / 14.54	74.38 / 13.53	73.52 / 15.08

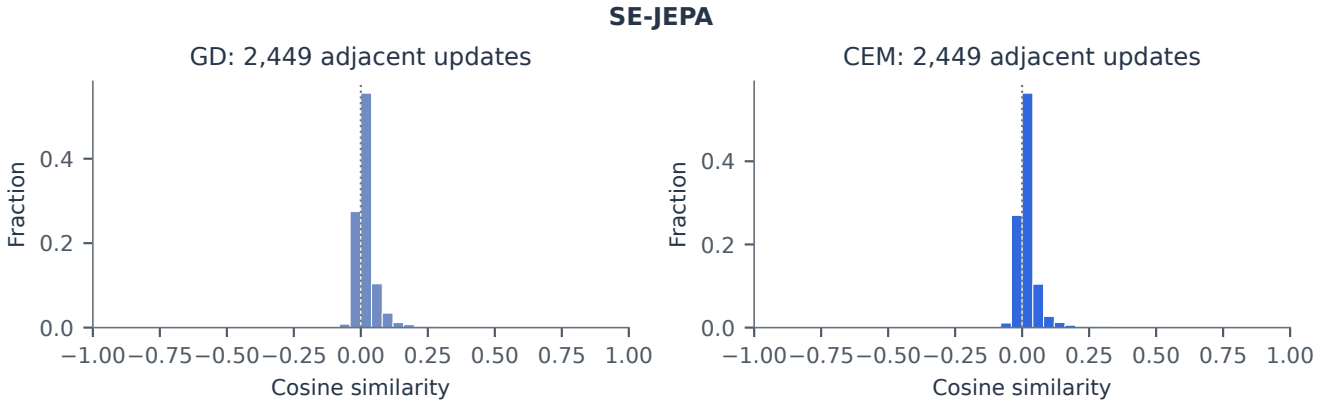


Figure 9. Cosine similarity between consecutive fast-update fingerprints in the seven-round shape stream (2,449 pairs per planner).

Intervals use 2,000 bootstrap resamples of the 50 physical tasks, retaining all visual conditions together. Among pairs eligible for retrieval, condition-averaged agreement with the future-best predictor is 82.7%. Five-step error falls from 0.04747 for the fixed-domain comparator to 0.03154 for selection, supporting reuse beyond the short prefix used for predictor selection.

E. Qualitative Examples and Reproducibility

E.1. Rigid-object planning sequences

Figure 3 selects two GD episodes where Frozen, DINO-WM, and AdaJEPA fail and SE-JEPA succeeds. The unseen-I example uses the paired shape evaluation. Its DINO-WM frame replays recorded actions; the same procedure exactly reproduces all 31 archived Frozen frames and states.

The red-object example comes from round five of the 14-domain stream, with the same initial configuration and goal for all methods. DINO-WM's final frame comes from the 30-replan simulation; the other final frames reproduce the recorded action sequences in the original simulator. Appendix D reports the mixed-stream evaluation.

Figure 3 labels MPC replans. All baseline runs fail after 30 replans of five environment actions, exhausting the same 150-action budget. SE-JEPA succeeds after six replans in each row. The examples were selected for these outcomes; aggregate success uses the complete evaluation sets.

Figures 10 and 11 show the square and red-agent CEM examples from the original execution archive. Both use the first held-out target of test seed 2000. We retain the full frame strips, termination markers, prediction-loss curves, and explanatory notes. The label Ours denotes SE-JEPA; Replay identifies frames reconstructed in the original simulator from the recorded actions. Gray frames hold the terminal state once the episode has reached its goal.

On the unseen square, SE-JEPA reaches the goal after five replans, compared with eight for AdaJEPA; Frozen exhausts all 30 replans without success. On the red-agent shift, SE-JEPA succeeds at replan ten while both baselines exhaust the budget. These cases show two benefits of memory: reaching a shared goal sooner and solving a goal that episode-local adaptation misses.

The plotted loss is prediction MSE over up to five re-

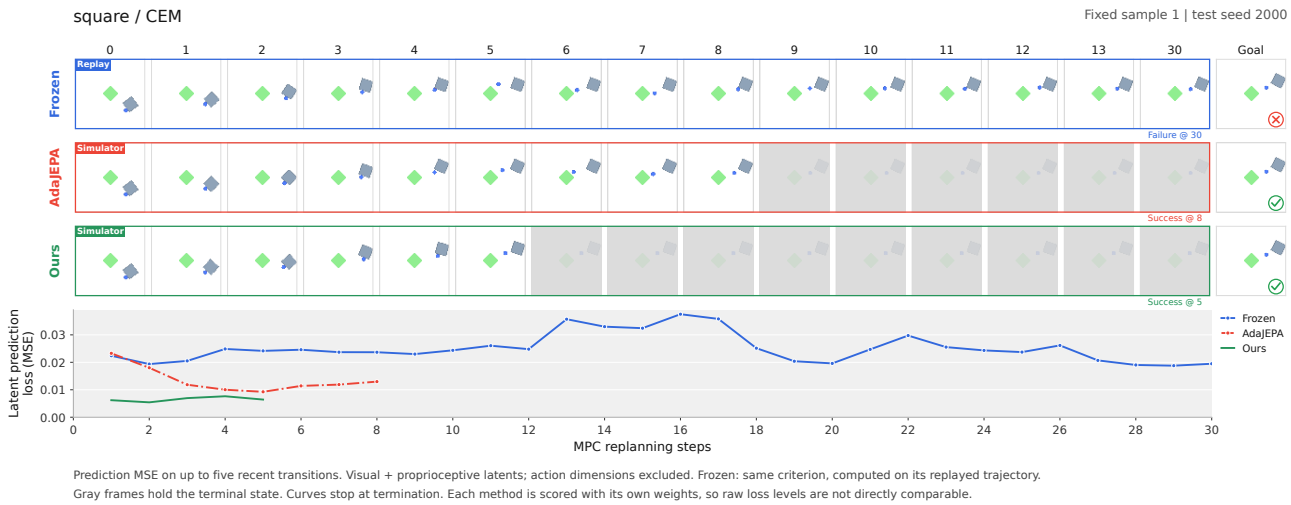


Figure 10. **Unseen-square planning under CEM.** The original trajectory strip includes replans 0–13 and 30, followed by the goal. SE-JEPA succeeds at replan five and AdaJEPA at replan eight; Frozen fails within 30 replans. Loss traces stop at termination. The in-panel notes specify frame provenance and the prediction-loss definition.

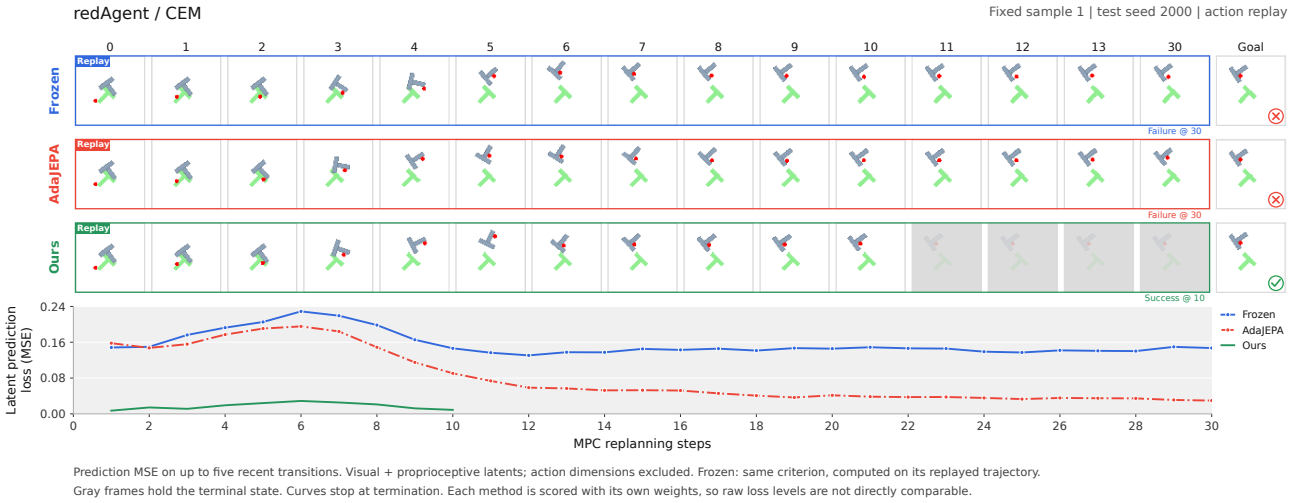


Figure 11. **Planning through a red-agent appearance shift under CEM.** SE-JEPA reaches the goal at replan ten; Frozen and AdaJEPA reach the 30-replan limit. The trajectory images are simulator replays of recorded actions. Falling prediction loss alone does not guarantee reaching the goal, as the AdaJEPA trajectory illustrates.

cent transitions in the visual and proprioceptive latents, excluding action dimensions. Each curve is evaluated with its own method's weights, so absolute loss levels are not directly comparable across methods. The curves show how each method's prediction error changes during execution; task success is the common measure of planning performance. In particular, AdaJEPA reduces its prediction loss on the red-agent episode but still misses the target, whereas SE-JEPA achieves the desired configuration.

E.2. Deformable rollout sequences

Figure 12 shows two complete deformable sequences: rope example 1 and granular example 2. Each row contains the start, every third push, and the goal within the same 15-push budget. These instances come from the repeated 50-instance stream and were selected where both baselines fail and the consolidated predictor succeeds. The examples use the third-pass slow predictor without within-episode adaptation; Table 1 reports fourth-pass aggregates. DINO-WM frames replay recorded actions in FleX; AdaJEPA and

SE-JEPA frames were saved during planning. Final CD values and success markers are retained from the original logs.

The first rope example starts at CD 1.72. Both baselines finish at 1.54, while SE-JEPA reaches 0.31 and the target success threshold. In granular example 2, final CD is 0.33 for both baselines and 0.14 for SE-JEPA. The sequences show how replayed experience improves the geometry of the final configuration under a common action budget, even with fast adaptation disabled.

E.3. Reproducibility

Source manifests link archived episode records, simulator images, outcomes, checksums, and crops. Appendix D specifies the mixed-stream evaluation. The release will include code, configurations, learned states, and the supporting evaluation records.

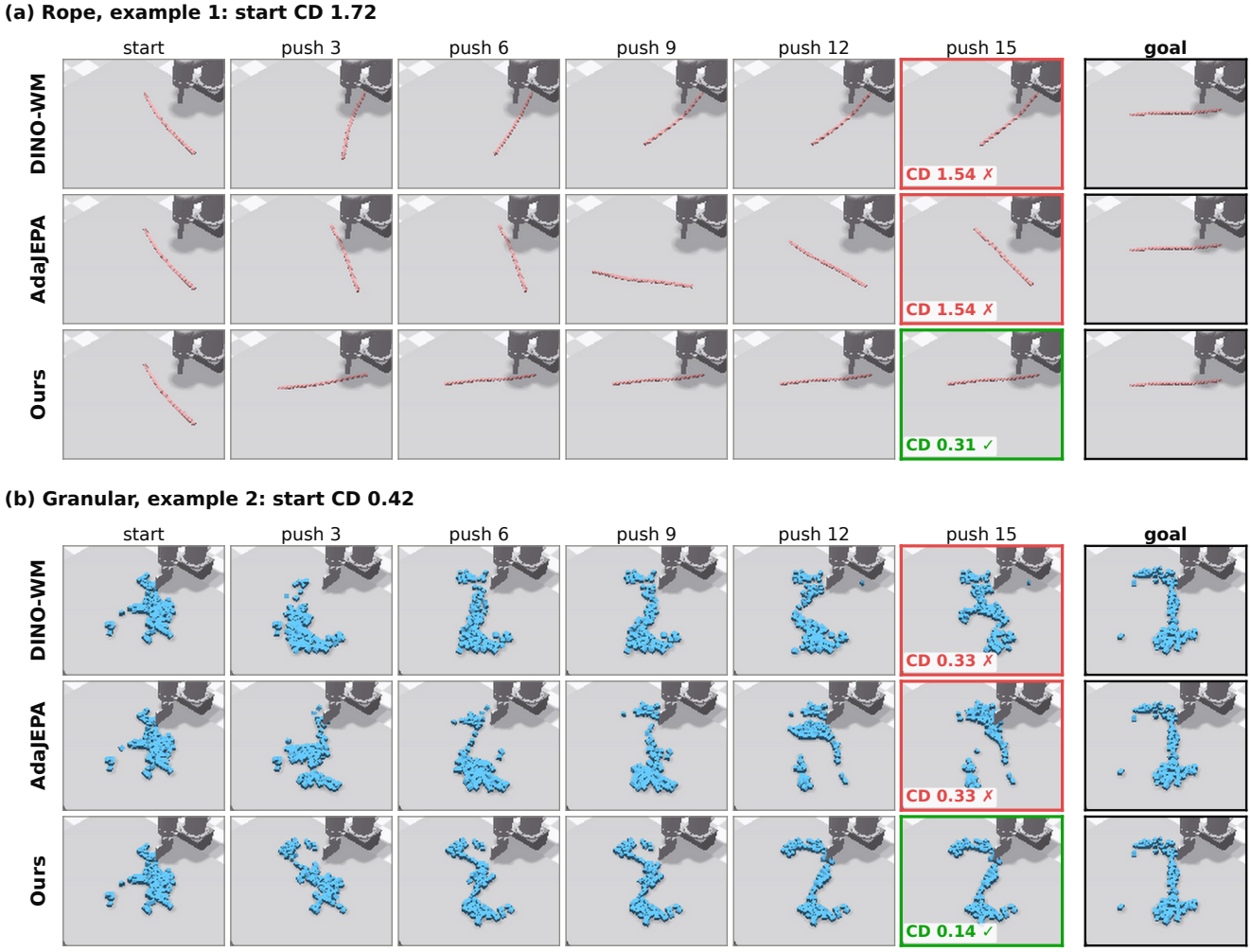


Figure 12. **Rope and granular execution sequences.** Top: rope example 1. Bottom: granular example 2. All methods share the initial configuration and goal. Columns show pushes 0, 3, 6, 9, 12 and 15, then the goal. Final-frame labels report CD (lower is better) and success. SE-JEPA reaches both goals with its consolidated slow predictor; both baselines fail.